CatSynth Artifact Supplement: The Full Sketch-Counterexample Run

Muness Castle
Independent
muness@muness.com

Eric Rubeck

July 2026

Use of generative AI. The CatSynth experiment used generative AI as documented below. OpenAI Codex also assisted with orchestration, editing, and revision; the authors reviewed the retained code, prompts, analyses, citations, and text and take responsibility.

This is the audit and reproduction companion to *Agentic Synthesis against Counterexample-Supplemented Sketches*. The paper contains the complete argument, method, experimental design, reported results, and limitations. The distributed paper appends this same material; `catsynth-supplement.pdf` packages it separately for readers who want the implementation trace on its own. Nothing in the supplement extends the paper's method or claims.

This supplement provides the implementation depth that would interrupt the paper: the complete CatSynth generation sequence, screenshots, reproduction commands, source map, and links from each counterexample to the revised sketch, code, prompt, and gate result. The browser makes the finished artifacts visible. The experiment harness asks a coding model to evolve the sketch, deterministic code, and model prompt one counterexample at a time.

CatSynth uses synthetic breed attributes and policy rows selected to make the control loop easy to inspect. Nothing here is pet-selection or medical advice.

1 The artifact boundary

For orientation, CatSynth maps the paper's method onto four operational responsibilities:

- The **operator** decides whether a failing case is an authoritative counterexample to the current sketch.
- The **evolved sketch** carries the policy learned from every accepted counterexample.
- The **CE archive** preserves every approved case and explains why the sketch changed.
- The **regression set** checks selected consequences against generated code.

The code and prompt are replaceable. A clean implementation should be regenerable from the evolved sketch and known-code anchors without replaying the CE archive as generation context.

CatSynth makes one simplifying choice: its run is small, so every accepted CE is also a regression case (`R = A`). That is why its gate sees every promoted case. The general method does not require the full archive to remain in the executable regression set.

The iterative arm also obeys one generation boundary:

> Developer sees one active failure. It never receives the CE archive or regression set as a bulk prompt.

The run starts from an initial sketch and empty `strategy.py` and `oracle_prompt.txt` files. Developer returns complete replacements for all three evolving artifacts:

```
SKETCH.md
strategy.py
oracle_prompt.txt
```

After the initial implementation passes its anchor, the harness reveals one proposed counterexample. It evaluates the case before approval. A case that already passes is coverage, not a counterexample; the harness records that result and continues to the next proposal without sending it to Developer.

A failing case becomes a CE only if it exposes missing sketch policy and the operator approves the corrected behavior. Developer then receives the current three files and that one failure. It must revise the sketch with the code or prompt. The gate runs the active case and current regressions. If an earlier case regresses, that failed regression becomes the next single Developer input. The harness reveals no new case until the gate is green.

The captured run freezes candidate cases and authoritative expected outputs before execution, then treats them as a simulated operator-approved stream. That makes the experiment reproducible, but it simulates the live approval step rather than giving the model authority to approve policy. Every accepted CE in the captured run changes `SKETCH.md`. The informational restriction prevents Developer from copying future counterexamples into the sketch because it never sees them.

2 Reproduce the run

From `examples/catsynth`:

```
uv run --with-requirements requirements.txt \
  python experiment/adaptive_open_world_experiment.py \
  --model gpt-5.4-mini \
  --max-repairs 12
```

This published three-path driver uses Codex App Server. The run used GPT-5.4-mini with low effort. The adapter disabled tools and environment access, disabled provider fallback, and used an ephemeral thread for every call. It consumed 173 model calls and 3,251,645 recorded model tokens across all three paths, including post-acceptance evaluation. Raw JSON-RPC transcripts stayed local; the repository retains every generated sketch, strategy, prompt, failure, gate outcome, diff, and usage total. The CatSynth README documents the portable driver for Codex App Server and OpenAI-compatible Chat Completions endpoints.

The captured evidence is in the checked-in run directory.

3 The starting point

The initial sketch fixes the public interface and the known preference ranking:

`recommend(profile, breeds, rules, oracle_tags)`

It defines the ordinal maps and exact preference weights, but deliberately leaves two policy surfaces open:

- rule rows exist, but the sketch does not yet say what matched rules do;
- `oracle_tags` exists, but no tag has a meaning yet.

That is the partial specification. It contains enough detail to generate and test an initial strategy without preloading the later policy discoveries.

The clean-room baseline contains no implementation. Both rebuild controls copy the same empty files at each discovery epoch.

4 Generation 000: the initial strategy

Developer generates the first complete sketch, deterministic strategy, and narrative prompt from the partial sketch and empty implementation files. The initial preference anchor passes:

```
initial-preference-ranking: PASS
gate: 1/1
```

Only then does the harness reveal the first domain counterexample. The complete generation is preserved under `arms/sketch-ce/generations/000-initial-generation/`.

5 Generation 001: hard policy must filter before ranking

The first profile describes an owner with mild allergies who wants a large, fluffy, affectionate cat. The current preference-only strategy returns Persian.

The approved counterexample requires:

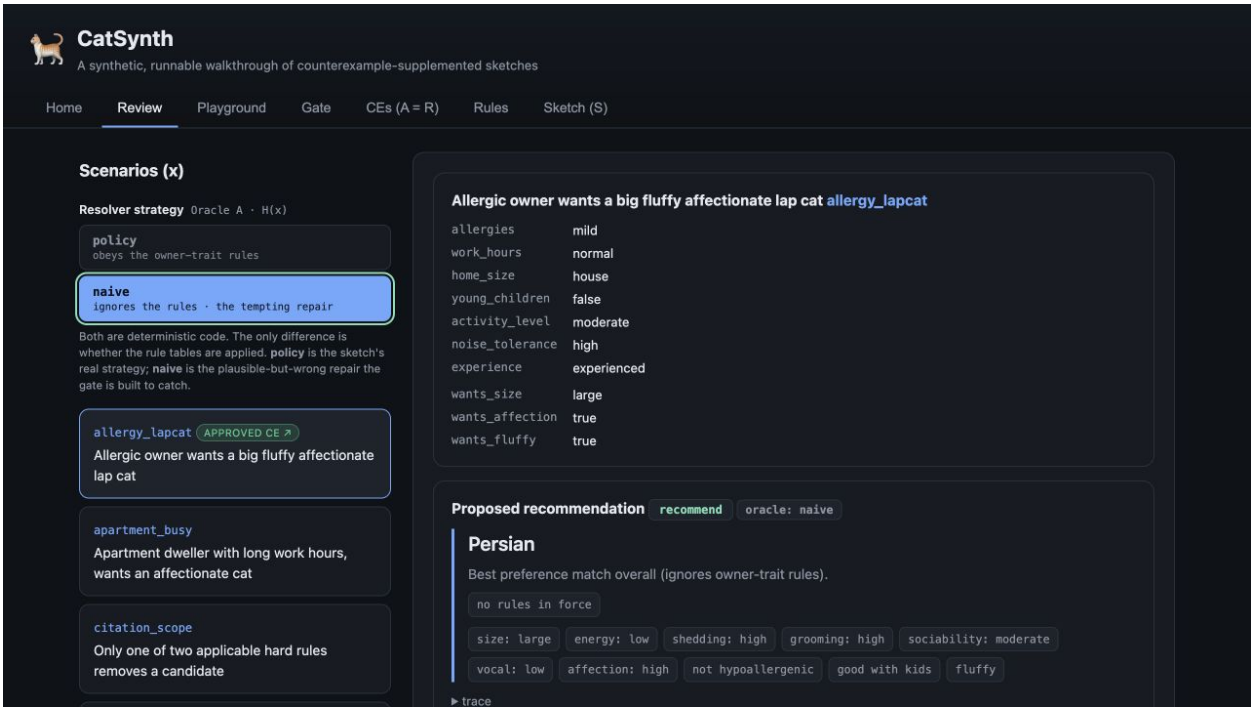
```
operation:    recommend
breed:        Siberian
cited_rules:  [allergy_requires_hypoallergenic]
```

The synthetic policy row says that mild or severe allergies activate a hard `forbid` rule for breeds whose `hypoallergenic` field is false. The correction adds a general ordering rule: filter hard-policy violations before ranking the survivors.

Developer receives only this failure and the current files. It revises the sketch and `strategy.py` to interpret the rule row generically. CatSynth's `R = A` gate then passes the initial anchor and CE1:

```
initial-preference-ranking: PASS
ce-001-allergy-override:    PASS
gate: 2/2
```

The browser presents the same near miss as a teaching surface:



The point is the encoded ordering, not the cat claim. Persian closes the visible preference gap; Siberian also closes it while satisfying the approved hard rule.

6 Generation 002: hard rules compose and may force abstention

The second counterexample activates five hard rules: severe allergies, apartment constraints, long work hours, and young children. The current implementation understands the first allergy operator but still returns Balinese and cites only that rule.

The approved expectation is:

```
operation: abstain
breed: null
cited_rules:
  - allergy_requires_hypoallergenic
  - apartment_no_high_energy
  - children_require_good_with_children
  - long_hours_no_high_sociability
  - severe_allergy_low_shedding
```

Developer generalizes again. The revised sketch and code support the additional profile and cat predicate operators, apply every applicable hard rule, and abstain rather than relaxing policy when no candidate remains.

The full regression passes:

```
initial anchor: PASS
CE1:            PASS
CE2:            PASS
gate:           3/3
```

7 Generation 003: the prompt learns a controlled tag

The third profile has no new structured policy row. Its missing information is in the narrative note:

```
I travel for work every few weeks and my last cat seemed miserable and lonely
whenever I was gone for days.
```

Before approval, the current prompt emits no tags and the deterministic strategy recommends Balinese. CE3 adds one controlled narrative classification to the evolved sketch:

- travel, repeated absence, or concern about loneliness maps to exactly `avoid_needy`;
- the prompt classifies only the supplied note and may not invent unrelated tags;
- the deterministic meaning of `avoid_needy` remains an open hole.

This case compares only `oracle_tags`. Developer revises the prompt and sketch. The gate passes 4/4 even though the strategy still chooses Balinese, because CE3 has not yet added a deterministic meaning for the tag.

```
initial anchor: PASS
CE1:            PASS
CE2:            PASS
CE3 tags:       PASS
gate:           4/4
```

8 Generation 004: a new counterexample closes the deterministic hole

After CE3, the prompt emits `avoid_needy`, but the strategy still recommends Balinese. The harness evaluates the next proposed case before approval. It fails on the breed field, so CE4 is a genuine new counterexample rather than another explanation pasted into CE3.

CE4 adds two connected rules:

- `avoid_needy` applies a one-point soft penalty to breeds with high sociability;
- base preference scoring continues to use only the three explicit `wants_*` fields: size, affection, and fluffiness. Default activity, noise, and experience values do not add score unless an approved sketch clause gives them semantics.

Developer revises the sketch and deterministic code. The prompt remains green. The gate passes 5/5:

```
initial anchor: PASS
CE1:            PASS
CE2:            PASS
CE3 tags:       PASS
CE4 ranking:    PASS
gate:           5/5
```

9 Generation 005: distinct soft rules compose

The next accepted case activates three different soft policy predicates. The current strategy returns Abyssinian because it stops short of applying the full soft-policy total. The approved output is British Shorthair.

CE6 adds a general rule: every distinct applicable `discourage` predicate contributes before the final ranking and tie-break. Developer revises the sketch and strategy; the complete gate passes 6/6.

10 Generation 006: duplicate concerns count once across surfaces

The next profile expresses the same high-energy concern twice: once through a structured policy row and once through a narrative note. Before repair, the prompt emits no tag. The approved output requires `avoid_high_energy`, while the ranking must apply the shared energy predicate only once.

CE7 makes the sketch join structured rules and narrative tags by the semantic triple of cat attribute, operator, and value. Developer revises the sketch, prompt, and strategy; the complete gate passes 7/7.

11 Generation 007: unknown safety data escalates

The next profile supplies `unknown` for allergy status. The current strategy treats it like no allergy and recommends Persian. CE10 distinguishes missing or unsupported safety data from a negative value: the implementation must return `escalate` with no breed until an operator can clarify the input.

Developer records the rule in the sketch and implements the escalation. The complete gate passes 8/8.

12 Generation 008: malformed applicable policy escalates with provenance

The final accepted case supplies an applicable hard policy row whose cat operator is unsupported. The current strategy silently ignores it and recommends Persian. CE12 requires `escalate` and cites `invalid_reviewer_policy`, preserving which policy source needs repair.

Developer adds the validation and provenance rule to the sketch and strategy. The full retained gate passes the initial anchor and all eight accepted counterexamples: 9/9.

The final retained Sketch-CE implementation passes 18/21 withheld cases. It misses two multi-tag cases because no accepted discovery has yet defined `avoid_vocal`, and it misses one normalized severe-allergy variant. Those failures show where another open-world counterexample could extend the current sketch.

13 What each generation archive proves

Every generation directory under `arms/` contains:

File	Evidence
`SKETCH.md`	Developer's complete revised strategy
`strategy.py`	Complete deterministic implementation
`oracle_prompt.txt`	Complete prompt implementation
`metadata.json`	Active failure or failures, accepted CE IDs, compact gate outcome, usage, and diffs

The Sketch-CE repair metadata identifies one active failure and no unrevealed case. The rebuild controls preserve each generated state and the visible failures returned after a failed gate. A reader can therefore inspect the information boundary and code evolution directly.

14 Audit questions

The retained histories let a reader answer five questions without relying on the paper's prose:

1. Which single failure was visible to each Developer generation?
2. How did the sketch, deterministic code, and prompt change together?
3. How did every accepted CE change the sketch?
4. Which regression gate ran after each revision?
5. Which proposed cases became accepted CEs, and which were recorded only as coverage?

15 The open-world comparison

The conceptual contrast remains spec-first versus Sketch-CE: a complete specification works when the problem is already known, while Sketch-CE changes the governing sketch as the world reveals new policy. The captured open-world experiment adds two controls to identify what carries that policy forward.

- **Replay-all** rebuilds from the initial sketch and every accepted case known at the current discovery epoch. It asks the model to infer policy again from raw examples.
- **Evolved-sketch rebuild** discards code and prompt, then receives only the current evolved sketch and known-code anchors. If its gate fails, it receives one visible failure at a time. This is the method's clean-regeneration test.
- **Sketch-CE with retained code** keeps the current sketch, code, and prompt and repairs each newly accepted case.

Measure	Replay-all	Evolved-sketch rebuild	Sketch-CE (retained code)
All recorded model tokens, including evaluation	**1,061,834**	**998,307**	**1,191,504**
Tokens through visible acceptance	891,880	828,628	1,021,822
Post-acceptance evaluation tokens	169,954	169,679	169,682
Developer calls	15	16	9
Developer tokens	400,081	371,050	217,576

Measure	Replay-all	Evolved-sketch rebuild	Sketch-CE (retained code)
Runtime Oracle tokens through acceptance	491,799	457,578	657,478
Specification Oracle tokens	0	0	146,768
Rebuilds	9	9	1
Extra repair attempts	6	7	0
Prior regressions on first attempt	2	7	0
Artifact churn lines	2,394	2,326	719
Final strategy LOC	224	228	298
Final decision nodes	77	70	110
Accepted CE evaluation	8/8	8/8	8/8
Withheld evaluation	15/21	19/21	18/21

The first row includes every recorded Developer, Runtime Oracle, Specification Oracle, and post-acceptance evaluation call. Tokens through acceptance exclude only the final visible and withheld evaluation. Provider totals count input plus output; cached input and reasoning are subsets, not additional tokens.

The candidate cases were external inputs. Sketch-CE paid to classify them and propose general rules for failures. Both controls inherited the promotion schedule, and evolved-sketch rebuild also inherited the sketch checkpoints. Their totals omit discovery work and are not end-to-end price rankings.

Sketch-CE with retained code used less Developer work and produced less cumulative churn. Evolved-sketch rebuild passed 19/21 withheld cases versus 15/21 for Replay-all. In this run, the evolved synthesis of the accepted examples generalized better than asking the same model to re-infer policy from all examples at every epoch. The retained strategy was the largest and had the most decision nodes, so that path shows less rework during evolution, not better final maintainability.

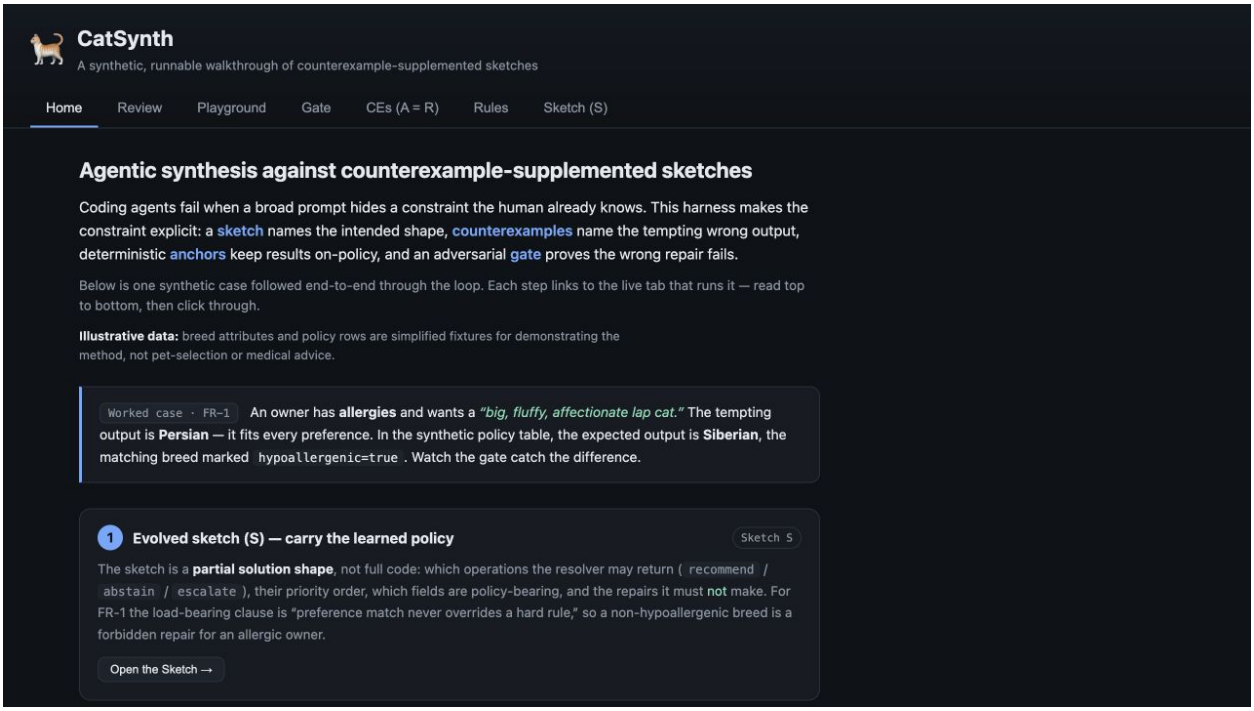
This is one model, one candidate order, and one sample per path. It supports the hypothesis that reviewed policy synthesis can carry lessons better than raw example replay. It does not establish universal cost, correctness, or maintainability superiority.

16 How the UI illustrates the finished artifacts

The experiment is the synthesis history. The browser is the finished inspection surface.

```
cd examples/catsynth
uv run --with-requirements requirements.txt python cli.py seed --no-wiki
uv run --with-requirements requirements.txt python cli.py serve
```

Open `http://127.0.0.1:8000`.

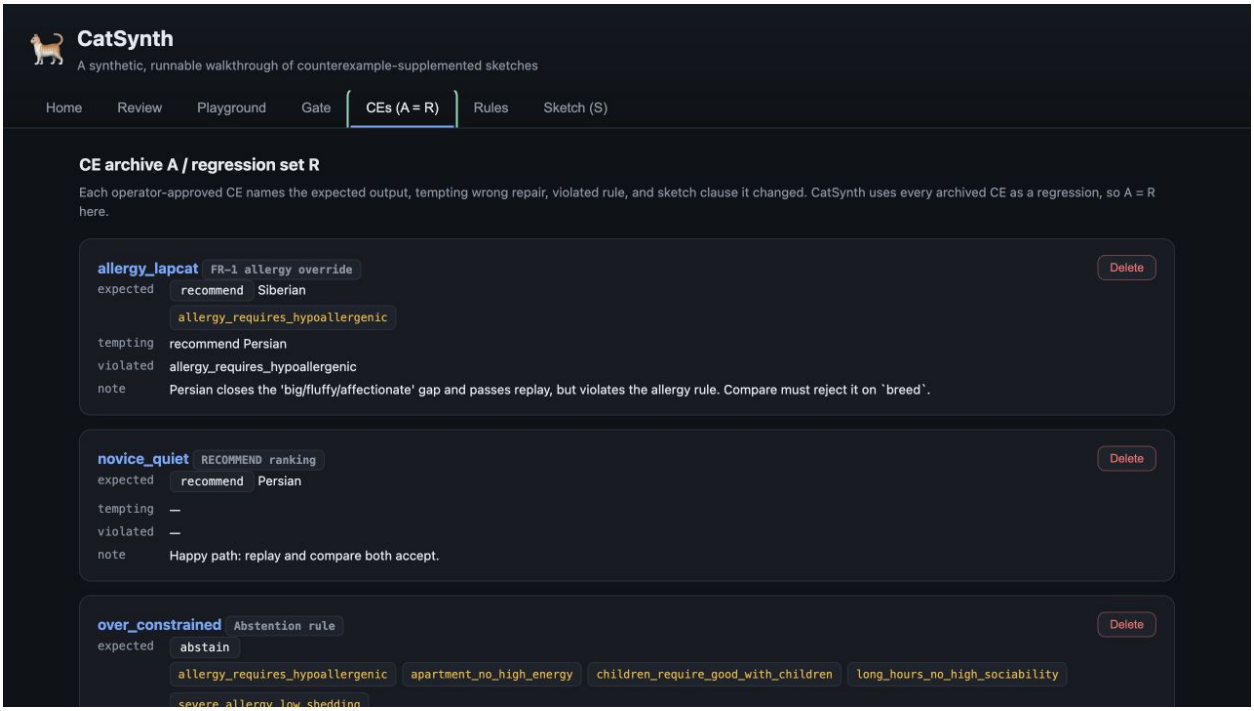


The UI exposes the stable repository artifacts:

1. **Sketch** - the strategy, policy order, holes, and abstention behavior.
2. **CE archive / regression set** - approved expected outputs, tempting outputs, violated rules, and sketch links. CatSynth shows the same cases in both roles because `R = A`.
3. **Oracle A** - deterministic hard-rule filtering, ranking, and abstention.
4. **Oracle B** - prompt-mediated narrative tags constrained to a controlled vocabulary.
5. **Gate** - replay and semantic comparison over CatSynth's regression set.

The browser's Review button records only a local operator decision in SQLite. It does not invoke Developer or revise `SKETCH.md`. The experiment history, not that button, is the evidence for the complete CE-to-sketch-and-code loop.

The CE archive / regression page keeps both sides of the focal correction:



An expected output records what should happen. The tempting output and violated rule record why a plausible alternative must continue to fail.

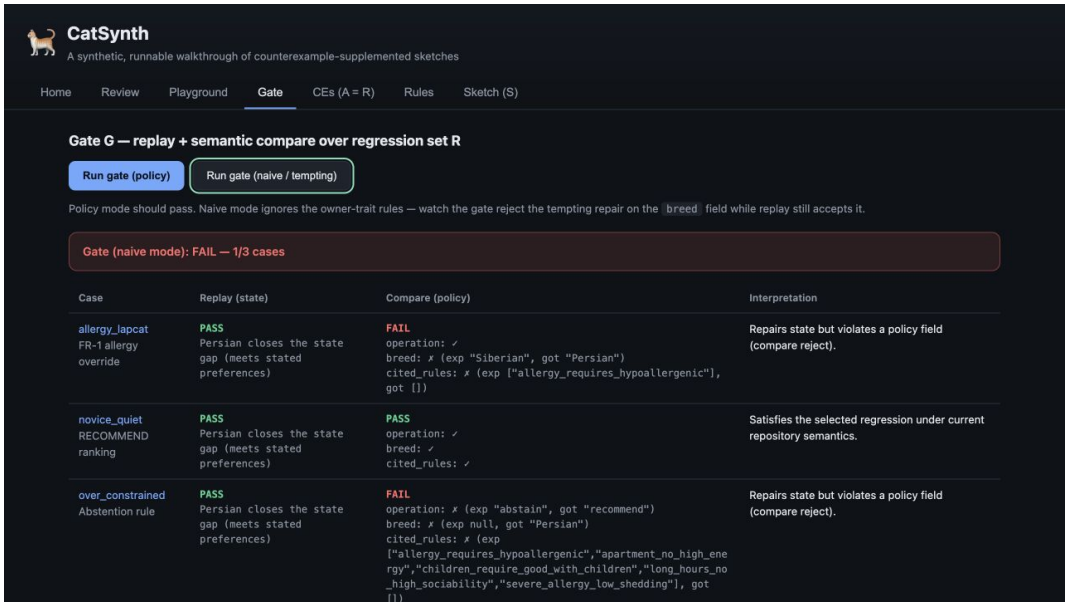
17 Why replay and semantic compare stay separate

Replay asks whether the candidate closes the visible state gap. For the focal recommendation, it checks the encoded size, affection, and fluffiness preferences. It deliberately does not decide the hard allergy policy.

Semantic compare checks the approved policy-bearing fields:

```
operation
breed
cited_rules
```

The naive Persian result can therefore pass replay and fail semantic compare:



The split makes the failure actionable. The candidate repaired the visible preference state but did not follow the approved policy.

18 Source map

Paper concept	Repository artifact
Initial sketch	`initial_sketch.md`
Candidate manifest	`adaptive_candidate_manifest.json`
Operator references	`cases.json`
Accepted CE archive and regression set (`R = A`)	`promoted-corpus.json`
Experiment driver	`run_experiment.py`
Codex App Server adapter	`codex_app_server.py`
OpenAI-compatible adapter	`openai_compat.py`
Oracle A	`oracle_a.py`
Oracle B	`oracle_b.py`
UI fixtures	`seed.py`
UI gate	`gate.py`
UI app	`app.py` and `static/`
Comparison harness	`adaptive_open_world_experiment.py`
Captured run	Published run

19 Claim boundary

CatSynth's iterative gate establishes finite-regression correctness only for these fixtures, expected outputs, and evaluators. The withheld cases provide limited evidence beyond the gate. Neither result establishes correctness for all owner profiles, real breed facts, bad expected outputs or checkers, unencoded policy, future model behavior, or other models and reveal orders. CatSynth preserves the evidence needed to inspect that boundary.